ACTIVIDAD DE APRENDIZAJE · SESIÓN 3

Actividad de Aprendizaje: La empresa de transporte

Practica herencia, polimorfismo y colecciones y exponlos en una API REST

Datos del participante	
Nombre	Alan Esteban Santana Mendoza
Fecha	19/10/26
Duración estimada	90 a 120 minutos
Modalidad	Individual (puede hacerse en parejas)
Entregable	Proyecto subido a GitHub + hoja de respuestas de este documento

Objetivos de aprendizaje

Al terminar esta actividad serás capaz de: v

- Proteger los datos de una clase con atributos privados y setters que validan.
- Crear una jerarquía de clases con extends, super y @Override.
- Usar polimorfismo para tratar objetos distintos de forma uniforme.
- Manejar colecciones: List/ArrayList y Map/HashMap.
- Recorrer, buscar y filtrar colecciones con for-each e if.
- Exponer colecciones de objetos como JSON en una API Spring Boot.

El escenario

Una empresa de transporte de la ciudad cuenta con buses, taxis y otros vehículos. Necesita una aplicación que registre su flota, calcule la tarifa de cada tipo de vehículo según los kilómetros recorridos, consulte quién conduce cada uno y muestre todo mediante una API REST. Tú construirás la primera versión.

> **Reglas de la actividad:** escribe el código tú mismo, ejecuta después de cada etapa y no avances hasta que la anterior funcione. Las Etapas 1 a 3 se hacen en Java puro (sin servidor); la Etapa 4 conecta todo con Spring Boot.

Mapa de etapas

Etapa	Qué harás	Tiempo	Puntos
1	Clase base Vehiculo con encapsulación	15 min	15
2	Herencia: Bus y Taxi con @Override	20 min	20
3	Colecciones en Java puro	20 min	15
4	Endpoints con colecciones	25 min	20
5	Pruebas y registro de resultados	10 min	10
6	Preguntas de comprensión	10 min	20
Reto	Mayor capacidad y suma total (opcional)	10 min	+10 extra

Etapa 1: Clase base Vehiculo

Genera un proyecto nuevo en start.spring.io (Maven, Java 21, Group com.ejemplo, Artifact transporte, dependencia Spring Web). En el paquete com.ejemplo.transporte crea la clase Vehiculo con estos elementos:

Elemento	Requisito
Atributos (private)	placa (String) y capacidad (int)
Constructor	Recibe placa y capacidad. Debe asignar la capacidad usando el setter para que se valide.
Getters	getPlaca() y getCapacidad()
setCapacidad(int capacidad)	Solo asigna el valor si es mayor que 0; en caso contrario lo ignora.
calcularTarifa(double km)	Devuelve km * 1500 (tarifa base por kilómetro).
descripcion()	Devuelve un texto con el formato: Placa ABC123, capacidad 4

> **Pista:** si tienes dudas con el setter, revisa la clase Animal de la guía de la Sesión 3. Recuerda que el constructor llama a setCapacidad(capacidad) en vez de asignar directamente.

Crea también la clase Practica con su main y prueba:

1. Crea un Vehiculo con placa "VEH301" y capacidad 8, e imprime su descripcion().
2. Intenta cambiar la capacidad a -5 con setCapacidad. Imprime la capacidad y comprueba que no cambió.
3. Cambia la capacidad a 10 y comprueba que sí cambió.

Verifica tu avance:

☐ Los atributos son private y el constructor usa el setter.

☐ Un valor negativo no modifica la capacidad.

☐ calcularTarifa(10) devuelve 15000.0.

Etapa 2: Herencia con Bus y Taxi

Crea dos subclases de Vehiculo que reutilicen sus datos y cambien la forma de calcular la tarifa:

Clase	Atributo nuevo	Comportamiento
Bus	ruta (String), con getRuta()	calcularTarifa devuelve km * 900. descripcion devuelve "Bus " seguido de la descripción del padre y ", ruta " + ruta.
Taxi	empresa (String), con getEmpresa()	calcularTarifa devuelve la tarifa del padre más 4000 de banderazo. descripcion devuelve "Taxi " seguido de la descripción del padre y ", empresa " + empresa.

> **Pista:** para reutilizar el cálculo o la descripción del padre usa super.calcularTarifa(km) y super.descripcion(). Cada constructor debe empezar con super(placa, capacidad). No olvides @Override en los métodos sobrescritos.

En el main de Practica prueba el polimorfismo:

1. Crea un Bus ("BUS101", 40, "Centro-Norte") y un Taxi ("TAX201", 4, "Taxis Bogotá"), ambos guardados en variables de tipo Vehiculo.
2. Imprime la descripcion() y calcularTarifa(10) de cada uno.
3. Completa la tabla de resultados.

Vehículo	descripcion()	calcularTarifa(10)
Vehiculo (VEH301)	Placa VEH301, capacidad 8	Tarifa 10km: 15000.0
Bus (BUS101)	Bus Placa BUS101, capacidad 40, ruta Centro-Norte	9000.0
Taxi (TAX201)	Taxi Placa TAX201, capacidad 4, empresa Taxis Bogotá	19000

Verifica tu avance:

☐ Bus y Taxi compilan y usan super(...) en el constructor.

☐ Cada vehículo muestra una tarifa distinta para los mismos 10 km.

☐ Puedes explicar por qué la variable es de tipo Vehiculo pero se ejecuta el método de Bus o Taxi.

Etapa 3: Colecciones en Java puro

En el main de Practica construye la flota y resuelve:

N.º	Tarea
1	Crea una List<Vehiculo> con ArrayList y agrega cinco vehículos: dos buses, dos taxis y un vehículo base (usa distintas placas y capacidades).
2	Con un for-each imprime para cada vehículo su descripcion() y su tarifa para 10 km.
3	Con un for-each e if cuenta cuántos vehículos tienen capacidad mayor a 10 e imprime el resultado.
4	Crea un Map<String, String> llamado conductores con tres pares placa → nombre del conductor (por ejemplo "BUS101" → "Carlos").
5	Imprime el conductor de una placa existente. Luego usa containsKey para comprobar una placa que no exista e imprime un mensaje adecuado.

> **Pista:** importa java.util.List, java.util.ArrayList, java.util.Map y java.util.HashMap (Alt+Enter en IntelliJ). Recuerda que las claves del mapa distinguen mayúsculas.

Verifica tu avance:

☐ La lista tiene 5 elementos (size() lo confirma).

☐ El recorrido muestra descripciones y tarifas diferentes según el tipo.

☐ Se maneja correctamente la búsqueda de una placa inexistente.

Anota cuántos vehículos tienen capacidad mayor a 10 en tu flota:

los dos buses, con capacidad 40 y 35

Etapa 4: Endpoints con colecciones

Crea la clase TransporteController con @RestController. La flota y el mapa de conductores deben ser atributos de la clase y llenarse en su constructor (con los mismos datos de la Etapa 3). Implementa estos endpoints y ejecuta TransporteApplication para probarlos:

Ruta	Debe hacer	Respuesta esperada
/vehiculos	Devolver toda la flota	Arreglo JSON de 5 vehículos
/vehiculos/placa/{placa}	Buscar por placa (sin distinguir mayúsculas) y devolver su descripción	Texto con la descripción o "No se encontró el vehículo ..."
/vehiculos/tarifa/{km}	Devolver una lista de textos con descripción y tarifa de cada vehículo	Arreglo JSON de 5 textos
/vehiculos/capacidad/{minima}	Devolver los vehículos con capacidad mayor o igual a la mínima	Arreglo JSON filtrado
/conductor/{placa}	Consultar el mapa de conductores	El conductor o "Sin conductor asignado"

> **Pistas:** para /vehiculos/tarifa/{km} declara @PathVariable double km, crea una List<String> vacía, recorre la flota y agrega a la lista un texto como descripcion() + " -> $" + tarifa. En /conductor/{placa} usa placa.toUpperCase() antes de consultar el mapa para que coincida con las claves.

Verifica tu avance:

☐ En /vehiculos los buses muestran su ruta y los taxis su empresa (cada clase muestra sus propios atributos).

☐ La búsqueda por placa funciona con mayúsculas y minúsculas.

☐ El filtro por capacidad devuelve solo los vehículos adecuados.

Etapa 5: Pruebas y registro

Prueba cada endpoint con el navegador o con Postman / Thunder Client y completa la tabla:

Petición (GET)	Código HTTP	Respuesta recibida (resumen)
/vehiculos		
/vehiculos/placa/bus101		
/vehiculos/placa/xyz999		
/vehiculos/tarifa/10		
/vehiculos/capacidad/10		
/conductor/bus101		
/conductor/xyz999		

Pregunta de análisis: reinicia la aplicación. ¿Qué pasa con los datos y por qué?

Al reiniciar la aplicación, todos los datos vuelven a su estado inicial porque están almacenados únicamente en la memoria RAM y aún no están conectados a una base de datos para tener persistencia.

Sube tu proyecto a GitHub y escribe aquí el enlace del repositorio:

https://github.com/Nicolascito/seccion3

Etapa 6: Preguntas de comprensión

Marca la opción correcta en las preguntas 1 a 4.

1. **¿Qué palabra clave permite que una clase herede de otra?**a) implements
   b) extends
   c) inherits
   d) super

2. ¿Qué indica la anotación @Override?

a) Que el método es privado
b) Que el método reemplaza al método con el mismo nombre de la clase padre
c) Que el método no devuelve nada
d) Que el método se ejecuta primero

3. ¿Qué colección es la más adecuada para buscar el conductor de un vehículo a partir de su placa?

a) ArrayList
b) String
c) HashMap
d) int

4. Si una lista tiene 5 elementos, ¿cuál es la posición del último?

a) 5
b) 4
c) 6
d) 0

5. Explica con tus palabras qué es el polimorfismo usando el ejemplo de los vehículos.

Es poder guardar distintos tipos de vehículos (como un Bus, un Taxi o un Vehiculo normal) dentro de una misma variable o lista de tipo Vehiculo, y que al llamar al mismo método como calcularTarifa(), cada uno haga su propio cálculo según lo que realmente es (el bus cobra a 900 el km y el taxi le suma los 4000 del banderazo).

---

6. ¿Qué hace super(placa, capacidad) en el constructor de Bus y por qué es necesario?

Llama al constructor de Vehiculo para inicializar la placa y la capacidad (pasando por la validación del setter) sin tener que repetir ese código en Bus. Es obligatorio ponerlo porque Vehiculo pide sí o sí esos dos datos para poder crearse.

Reto opcional: el vehículo más grande

Crea un endpoint /vehiculos/mayor-capacidad que devuelva el vehículo con más capacidad de la flota.

Ruta	Respuesta esperada	Técnica
/vehiculos/mayor-capacidad	JSON del vehículo con mayor capacidad	for-each con variable que guarda el mayor encontrado

> **Pista:** empieza con Vehiculo mayor = flota.get(0); recorre la flota y, si el vehículo actual tiene más capacidad que mayor, reasígnalo. Al terminar el ciclo devuelve mayor.

Reto extra: crea /vehiculos/total-capacidad que devuelva la suma de las capacidades de todos los vehículos, con el formato "Capacidad total de la flota: N pasajeros".

Autoevaluación y rúbrica

Marca lo que lograste:

☐ Puedo explicar la diferencia entre herencia y polimorfismo.
☐ Escribo subclases con super y @Override sin copiar el código.
☐ Sé elegir entre List y Map según el problema.
☐ Recorro y filtro colecciones con for-each e if.
☐ Probé todos los endpoints y completé la tabla de resultados.
☐ Subí el proyecto a GitHub.

Criterio	Puntos máx.	Obtenidos	Observaciones
Etapa 1: clase Vehiculo con encapsulación	15		
Etapa 2: herencia, super y @Override	20		
Etapa 3: colecciones en Java puro	15		
Etapa 4: endpoints funcionando	20		
Etapa 5: pruebas y registro	10		
Etapa 6: preguntas de comprensión	20		
Reto opcional	+10		
TOTAL	100		

Resultados y respuestas

Elemento	Resultado esperado
Tarifa para 10 km	Vehiculo: 15000.0 | Bus: 9000.0 | Taxi: 19000.0
Vehículos con capacidad mayor a 10	2 (los dos buses)
/vehiculos/placa/bus101	Bus Placa BUS101, capacidad 40, ruta Centro-Norte
/vehiculos/capacidad/10	Los dos buses (capacidades 40 y 35)
/conductor/bus101	El conductor de BUS101 es Carlos
/vehiculos/total-capacidad	Capacidad total de la flota: 91 pasajeros (40 + 35 + 4 + 4 + 8)

Pregunta	Respuesta
1	b) extends.
2	b) Que el método reemplaza al método con el mismo nombre de la clase padre.
3	c) HashMap (permite buscar por clave).
4	b) 4 (las posiciones empiezan en 0).
5	Una misma llamada (por ejemplo calcularTarifa) produce resultados distintos según el objeto real (bus, taxi o vehículo base), aunque la variable sea de tipo Vehiculo. Se acepta cualquier explicación equivalente.
6	Llama al constructor de la clase padre para que se inicialicen placa y capacidad (y se aplique la validación del setter). Es necesario porque Vehiculo no tiene constructor sin parámetros.

Notas para la evaluación

- **Reinicio de la aplicación (Etapa 5):** los datos vuelven a su estado inicial porque viven solo en memoria. Es el gancho para introducir la persistencia con base de datos en sesiones posteriores.
- **/vehiculos/placa/xyz999 y /conductor/xyz999:** ambos devuelven código 200 con un mensaje de texto, aunque el recurso no exista. Pregunta a los participantes si es lo ideal; en la Sesión 4 se verá cómo responder con códigos como 404.
- **Tarifa del Taxi:** con super.calcularTarifa(10) + 4000 el resultado es 19000.0. Si algún participante obtiene otro valor, revisa que no esté sumando los 4000 a una tarifa ya modificada.
- **JSON de /vehiculos:** calcularTarifa y descripcion no aparecen porque no son getters. Los buses incluyen "ruta" y los taxis "empresa".
- **Errores comunes:** olvidar super(...) en el constructor, escribir mal un método sobrescrito (el @Override ayuda a detectarlo), usar == para comparar placas, pedir get(5) en una lista de 5 elementos o buscar claves del mapa con distinto uso de mayúsculas.